

基于 RK3588 的实验室多路智能安防人员管理系统与零拷贝优化

摘要

本项目以瑞芯微 RK3588 为核心平台，针对实验室人员与设备管理需求，搭建了一套多路智能安防系统。系统能同时处理多路 1080p 视频流，完成人员检测、人脸识别、设备供电控制等功能。

软件上采用四进程协同架构。主进程负责视频拉流和 YOLO 推理，用 6 个工作线程绑定 NPU 三个核心并行处理。人脸识别进程基于 InspireFace 框架运行，通过 V4L2 采集 USB 摄像头画面或拉取 ESP32 RTSP 流做识别。设备进程内 DeviceManager 用 miio 协议管米家插座，Esp32Manager 用 MQTT 协议管 ESP32 摄像头。QT 进程用 OpenGL 渲染 8 路画面到 HDMI 显示器。

在数据通路上，系统用到了 RK3588 的 MPP 硬件解码器、RGA 图形加速器和 NPU。从视频解码到色彩转换、推理、显示、录像，全程通过 DMA-BUF 文件描述符传递同一块物理内存，不需要 CPU 搬运数据，把延迟压到了毫秒级。

系统接入了 ESP32-S3 摄像头作为被动识别入口。用户在设备前站定后，设备进程通过 MQTT 通知 ESP32，ESP32 等待物理按钮确认后推流给人脸识别进程，识别通过即通过 miio 协议开插座并启动倒计时，时间到自动断电。

测试结果表明，系统支持 8 路 1080p 实时处理和显示，人脸识别在 25ms 以内完成，插座控制响应在 500ms 以内。每个单核 CPU 占用在 50% 左右，四进程内存合计约 800MB 左右，能够稳定运行。

1、 作品概述

1.1 功能与特性

系统的核心功能可以分成几部分：

一是多路视频采集与显示。通过 GStreamer 拉取多路 RTSP 流，经 MPP 硬件解码后送 OpenGL 渲染到 HDMI，同时在画面上叠加 YOLO 检测框和 ROI 状态色框。

二是人员检测与 ROI 判定。主进程用 6 个 worker 线程绑定 NPU 三核做 YOLOv5 推理，结合 ROI 区域判断人员停留时间，超过 3 秒触发设备登记流程。

三是人脸识别身份核验。基于 InspireFace 在 NPU 上运行，支持 USB 摄像头签到签退、设备登记、人脸录入三种场景，5 帧连续确认保证准确率。

四是智能插座控制。设备进程内 DeviceManager 通过 miio 协议直连米家插座 3，支持开关、软件倒计时、过载过温保护，默认断电保证安全。

五是 ESP32 被动识别。Esp32Manager 通过 MQTT 调度 ESP32 摄像头，用户按按钮确认后推流识别，30 秒超时自动取消。

六是录像管理，基于 FFmpeg 硬件编码，支持事件触发录像保存在本地 sd 卡中。

七是 QT 双屏管理界面，HDMI 显示 8 路画面可叠加显示检测框以及 ROI 设备区域，DSI 触摸屏提供设备管理，ROI 配置，记录查询，签到签退设备使用登记和人脸录入，以及录像回放查询和系统状态查看。

1.2 应用领域

系统主要面向高校和科研院所的实验室场景。

在实验室里，系统可以实现刷脸登记、自动供电、计时使用、超时断电的完整闭环。人员走到设备前，识别通过后插座自动上电，到达设定时长后自动断电。这样既解决了设备无人看管时的用电安全隐患，也能量化每台设备的使用时长。

除了实验室，系统也适用于共享实验室、创客空间、机房、精密仪器室这类需要身份核验和设备授权的场合。人脸签到签退功能可以直接用于人员出勤记录和工时统计。

系统的多路视频监控和智能检测能力，也能用在园区安防、仓库监控等传统

场景。零拷贝架构和多进程设计保证了良好的可扩展性，后续可以移植到工业质检、智慧零售等领域。

1.3 主要技术特点

(1) 全链路零拷贝。视频从 MPP 解码、RGA 色彩转换、NPU 推理到 OpenGL 显示和硬件编码录像，全程通过 DMA-BUF fd 传递同一块物理内存，没有 CPU 数据拷贝。

(2) 四进程协同架构。四个进程各司其职，通过 Unix Socket 通信，职责隔离也故障隔离。数据共享用 SQLite 的 WAL 模式，多进程并发安全。

(3) NPU 三核并行推理。6 个 worker 线程按 worker_id%3 绑定 NPU 三个核心，配合 MPMC 无锁队列（容量 256）实现生产者-消费者流水线。

(4) Tag-based IPC 协议。进程间通信用 4 字节标签加 payload 的格式，配合非阻塞 send 和 try_lock，实现 fire-and-forget 异步通信，推理线程不会因为 IPC 阻塞。

(5) 多下位机异构协议管理。设备进程内 DeviceManager 用 miio 协议管插座、Esp32Manager 用 MQTT 协议管 ESP32，两种协议各管各的设备。

1.4 主要性能指标

表 1-1 主要性能指标表

指标	参数
视频流路数	支持 8 路 1080p@30fps
YOLO 推理帧率	总吞吐约 150fps（6 线程 • 3 核 NPU） （采用跳帧推理）
人脸识别耗时	<25ms（单帧）
IPC 端到端延迟	<1ms（本地 Unix Socket）
插座控制响应	<500ms（miio LAN）
单核 CPU 占用率	<=50%（零拷贝优化后）
内存占用	<800MB（四进程总计）

1.5 主要创新点

(1) 全链路零拷贝管线：MPP 解码→RGA→NPU→OpenGL→硬件编码全程 DMA-BUF fd 传递，消除 CPU 数据拷贝。

(2) fire-and-forget 异步 IPC：Tag 协议加非阻塞 send 加 try_lock，推理线程不会因 IPC 阻塞。

(3) 多下位机异构协议协调：DeviceManager 用 miio 管插座、Esp32Manager 用 MQTT 管 ESP32，协同实现人脸登记到自动供电的闭环。

(4) peek-then-consume 鲁棒协议：解决了 Unix Socket 多消息合并导致的协议失步问题。

(5) 自实现米家智能插座 3 协议：不依赖米家官方 SDK，基于 python-miio 协议规范用 C++ 从零实现 miio 通信（UDP 54321 + AES-128-CBC 加密 + MD5 校验 + JSON-RPC），实现插座开关、功率监测、过载保护、倒计时等全部功能的本地化自主控制。

1.6 设计流程

项目按模块化增量方式推进，先做需求分析和整体方案设计，明确四进程架构和通信协议；然后搭建硬件平台，部署 RK3588 环境和外设连接；接着按优先级开发核心模块，先打通 GStreamer 拉流和 MPP 解码，再集成 YOLO 推理和 ROI 检测，然后做多进程 IPC 和人脸识别，最后集成设备进程和 ESP32 协调，全程用 ESP32 模拟器配合联调。



图 1.1 设计流程图

2、 系统组成及功能说明

2.1 整体介绍

系统以 RK3588 为核心主控，采用四进程加多下位机的分布式架构。

RK3588 上跑四个软件进程：QT 界面进程、主进程、人脸识别进程、设备进程。外部硬件节点包括 RTSP 摄像头、ESP32-S3 摄像头、米家智能插座 3 和双屏显示器（HDMI 加 DSI）。

进程之间用 Unix Socket 做本地 IPC 通信。设备进程内有两个管理模块：DeviceManager 用 miio 协议直连智能插座，Esp32Manager 用 MQTT 协议调度 ESP32 摄像头。所有进程共享同一个 SQLite 数据库，用 WAL 模式保证多进程并发安全。

系统整体架构如下图所示。

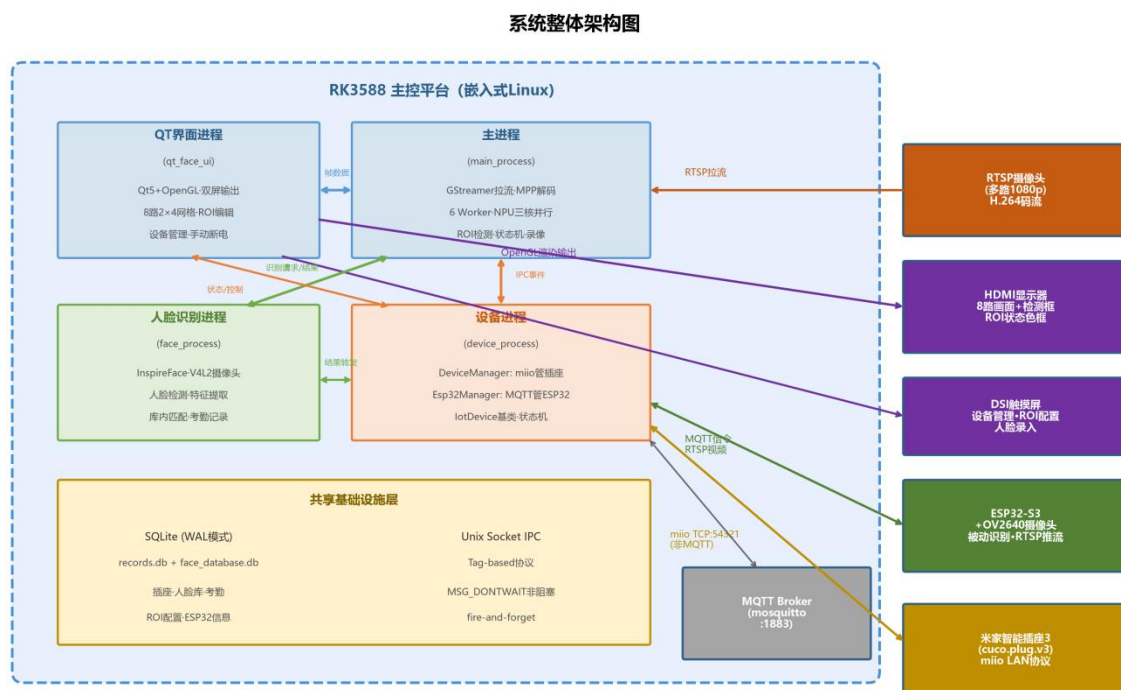


图 2.1 系统整体架构图

2.2 硬件系统介绍

2.2.1 硬件整体介绍

整个系统由PD诱骗供电、风扇散热以及ESP32控制交互三大环节构成，形成一条从USB-C取电、经诱骗调压后驱动风扇为RK2588降温，同时由ESP32拓展板直连TFT触摸屏以实现界面控制和状态反馈的完整闭环链路。

2.2.2 机械设计介绍

整体封装壳子设计由上壳，下壳和内部支撑壳子构成，如图2.2所示。内部结构剖面图结构如图2.3所示。

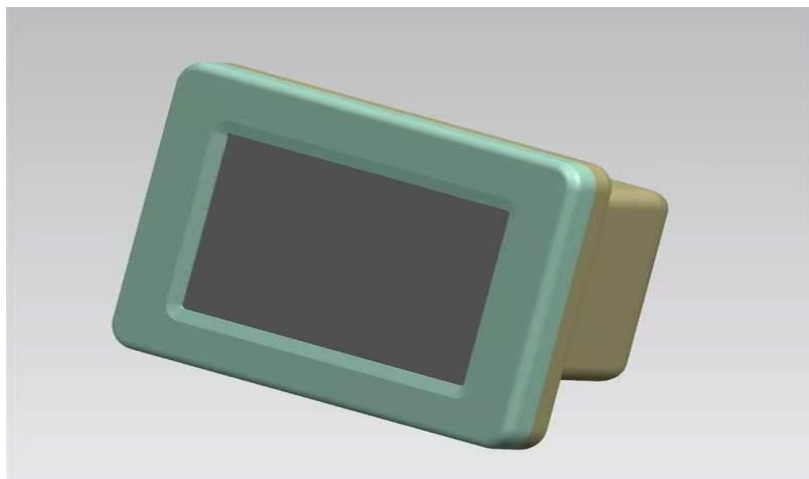


图 2.2 总体三维外观图

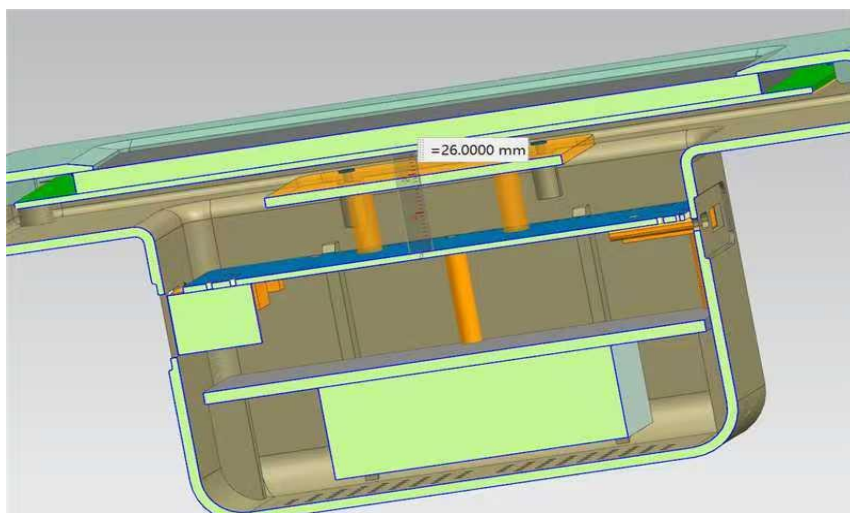


图 2.3 内部结构图

显示屏封装在上壳中，上壳对显示屏进行包边处理，设计结构如图 2.4 所示。



图 2.4 上壳图

设计支撑板子，用来支撑固定显示屏和驱动显示屏板子的中间区域，并在中间挖槽，便于显示屏和驱动板子之间引线进行连接。结构如图 2.5 所示。

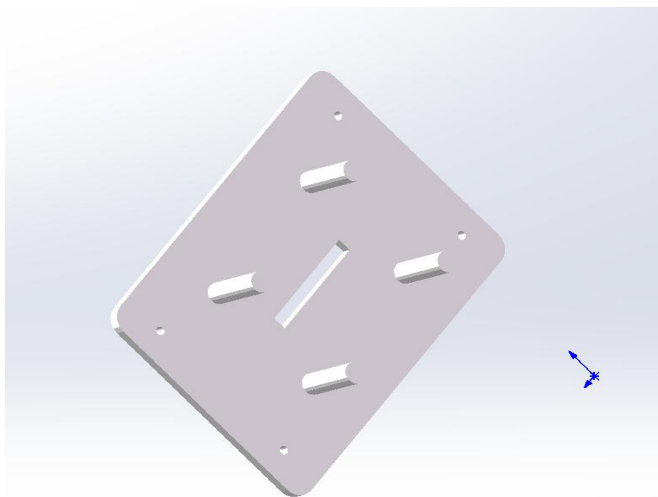


图 2.5 支撑板子图

PCB 板子封装在下壳中，下壳由上壳的底部壳子和 PCB 板子的封装壳子构成，在下壳中对板子相关的接口进行挖槽开口，同时考虑到板子主控芯片的散热问题，在下壳底部做镂空处理，如图 2.6，图 2.7 所示。

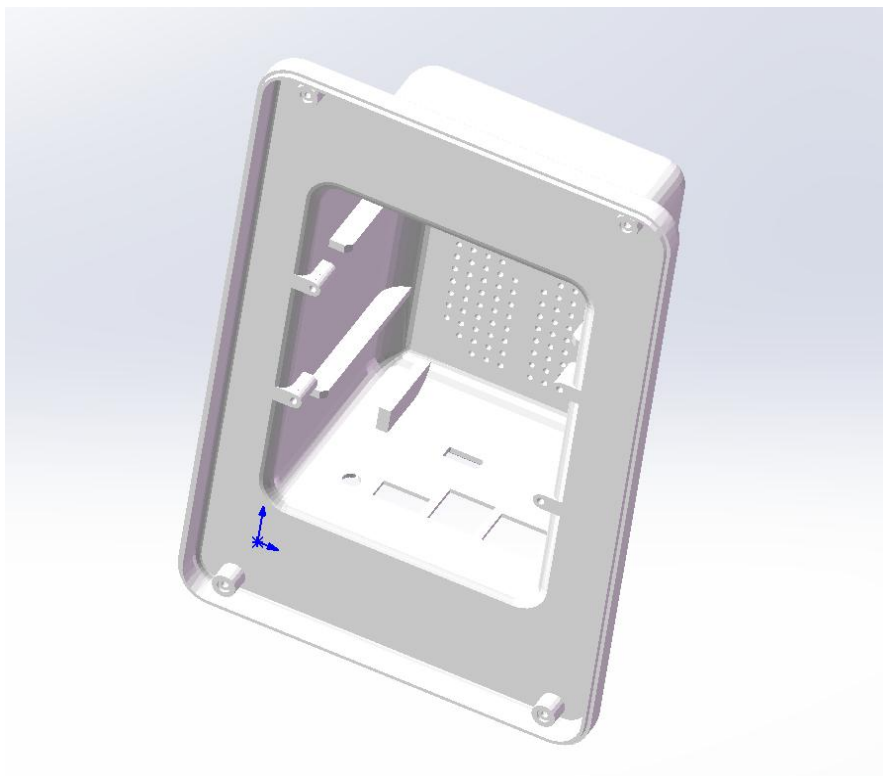


图 2.6 下壳图

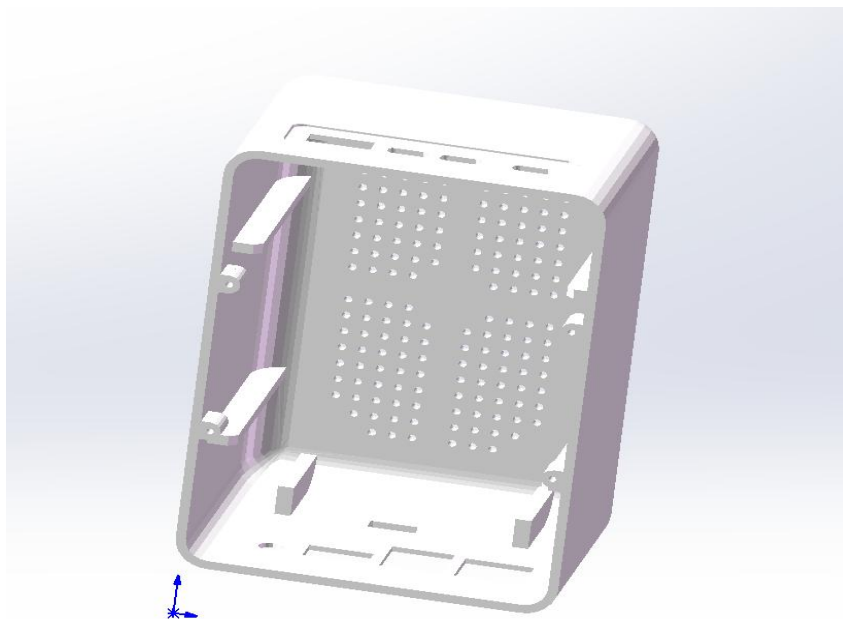


图 2.7 PCB 板子封装外壳图

2.2.3 电路各模块介绍

PD 诱骗器模块采用 CH224K 作为核心协议芯片，负责与 USB-C PD 充电器通过 CC 线进行实时通信，主动向充电器协商并“诱骗”出稳定的 12V 电压，为散热风扇提供充足动力；该模块外围电路极为精简，通过拨码开关即可灵活配置目标电压档位，并在电源输入输出端增设 MLCC 滤波电容，以确保大电流输出时的电压纯净度和带载稳定性。

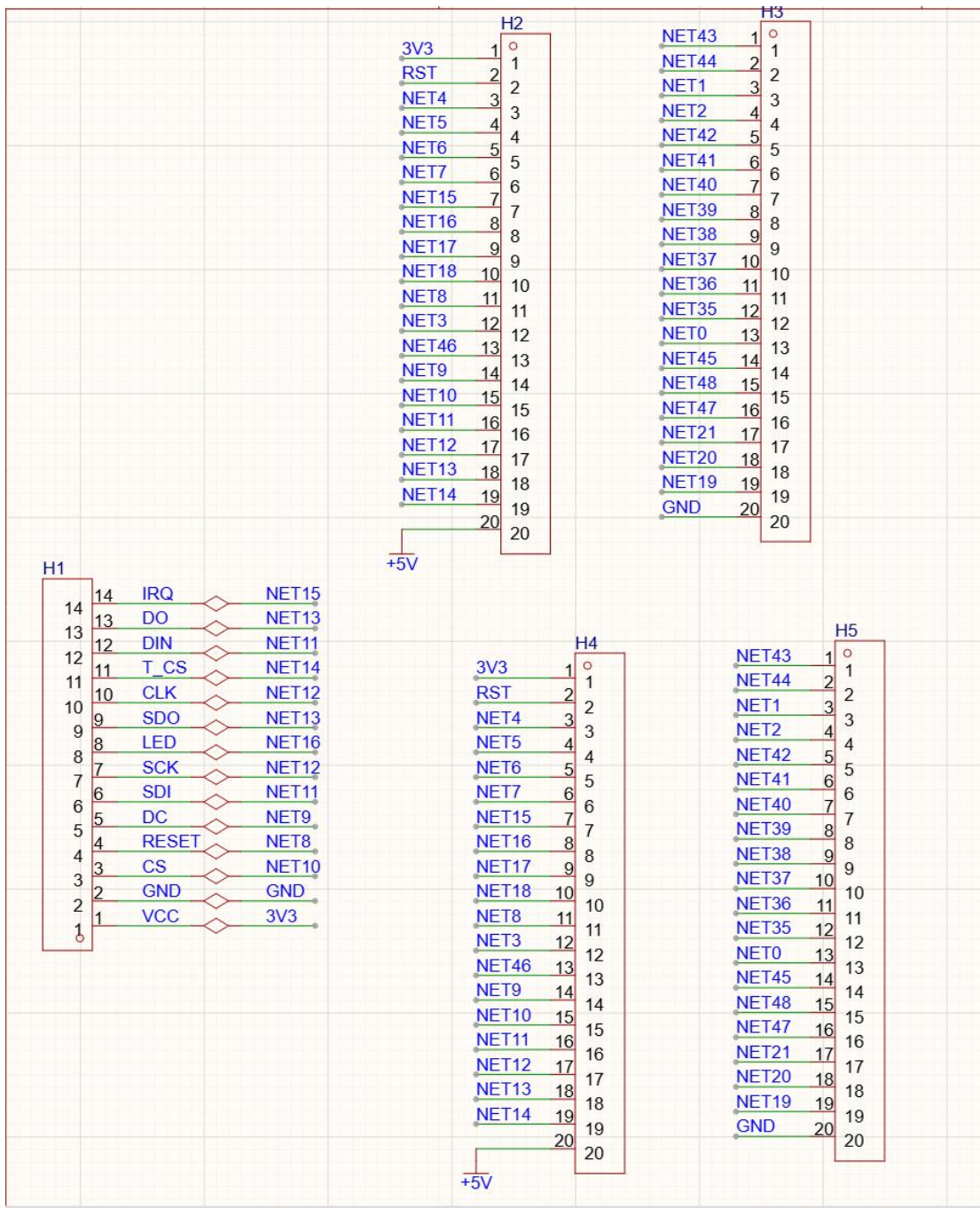


图 2.8 引脚图

ESP32 拓展板的设计初衷是摒弃杂乱的杜邦线飞线，通过 PCB 板级直接走线将主控与 TFT 触摸屏进行物理对接，从根源上减少高速 SPI 通信时的信号反射与空间耦合干扰；板卡上合理规划了屏幕显示和触摸控制所需的全部关键引脚（包括时钟、数据、片选及中断信号），并严格优化了等长布线规则，有效保证了屏幕在高刷新率场景下的数据传输完整性与触摸响应的实时精准度。

2.3 软件系统介绍

2.3.1 软件整体介绍；

软件系统部署在 RK3588 的嵌入式 Linux 环境上，采用四进程协同架构，各进程职责明确、故障隔离。

主进程是系统核心，负责多路视频流采集、MPP 硬件解码、YOLO 目标检测、ROI 区域判定、状态机管理和录像调度。它通过 MPMC 无锁队列把采集到的帧分发给 6 个工作线程并行推理，worker 按 worker_id 取模 3 绑定 NPU 三个核心。

人脸识别进程基于 InspireFace 框架运行在 NPU 上。它通过 V4L2 采集 USB 摄像头画面做签到签退，也能拉取 ESP32 RTSP 流做被动识别。用 FaceModule 的 mutex 保证 NPU 串行访问，5 帧连续确认同一人才算识别成功。

设备进程作为物联网网关，内部有两个独立管理模块。DeviceManager 通过 miiio 协议直连米家插座 3，插座设备继承自 IotDevice 基类。Esp32Manager 通过 MQTT 协议调度 ESP32 摄像头，不继承 IotDevice，是独立的任务调度器。两个模块用不同协议管不同设备。

QT 界面进程基于 Qt5 加 OpenGL 实现，支持双屏输出。HDMI 屏显示 8 路监控画面（2×4 网格或全屏），DSI 触摸屏提供设备管理和 ROI 配置界面。通过 Unix Socket 接收主进程推送的 DMA-BUF fd 做零拷贝渲染。

四个进程通过 Unix Socket 通信，采用标签前缀协议确保消息边界，配合非阻塞 I/O 和 try_lock 实现 fire-and-forget 异步通信。软件模块架构如下图所示。

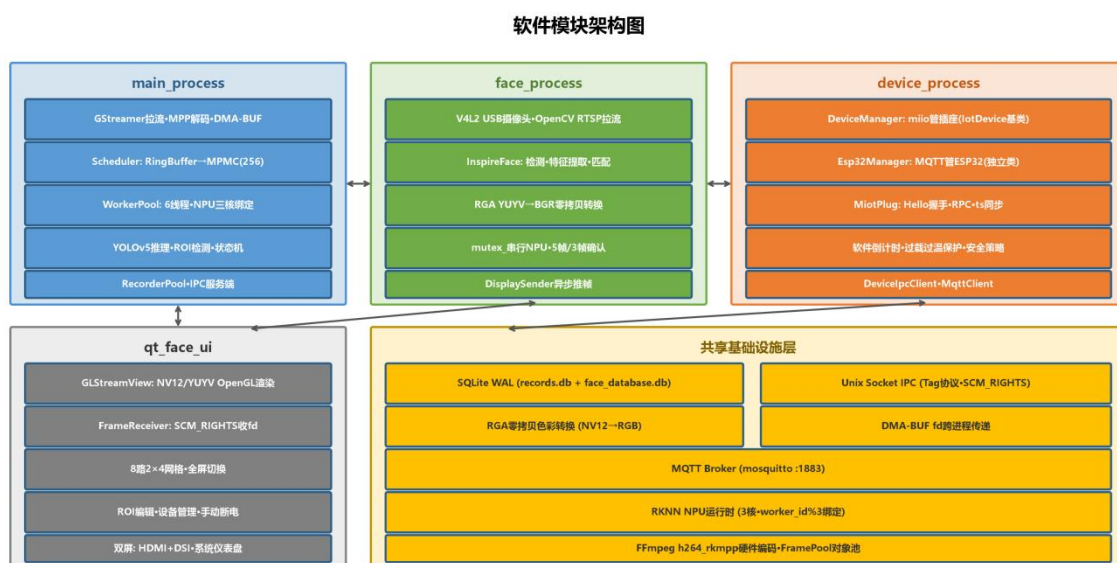


图 2.9 软件模块架构图

2.3.2 软件各模块介绍

(1) 主进程 (main_process)

主进程是系统的核心编排者，包含以下关键模块：

① GStreamer 采集与 MPP 解码模块

用 GStreamer 管道采集多路 RTSP 视频流，pipeline 为 rtspsrc 加 rtpH264depay 加 h264parse 加 mppvideodec。MPP 硬件解码器输出 NV12 格式的 DMA-BUF 缓冲区，通过 `gst_dmabuf_memory_get_fd` 取出文件描述符，后续模块全程通过 fd 传递，不需要 CPU 拷贝。每路流独立维护解码器实例，支持断流自动重连。

② FramePool 对象池与 Scheduler 模块

FramePool 预分配 6 个 Frame 对象 (`shared_ptr` 加自定义 deleter)，避免频繁分配释放。Scheduler 从每路 SPSC RingBuffer 取帧后 `try_enqueue` 到 MPMC 队列 (容量 256)，队列满时丢帧不阻塞生产者。

③ WorkerPool 推理模块

调度 6 个工作线程从 MPMC 队列取帧做 YOLOv5 推理。每个 worker 按 `worker_id` 取模 3 绑定 NPU 三个核心，通过 `rknn_dup_context` 共享模型权重但拥有独立的输入输出内存。采用跳帧策略 (每路每 2 帧推理 1 帧) 平衡推理负载和显示帧率。

④ RGA 色彩转换模块

用 RGA 硬件加速器把 NV12 格式转换成 NPU 需要的 RGB888 格式。通过 `importbuffer_fd` 把 DMA-BUF fd 注册到 RGA，`improcess` 执行转换加 letterbox 缩放，数据直接写入 NPU 输入内存，不需要 CPU 参与。

⑤ ROI 检测与状态机模块

对 YOLO 检测结果做 ROI 区域判定，维护每路每设备的状态机。状态流转为：空闲到人进入到停留超阈值触发登记到识别成功设免打扰到免打扰到期回到空闲。ROI 配置支持运行时热重载。

⑥ RecorderPool 录像模块

每路流一个独立录像线程加 MPMC 队列。基于 `shared_ptr` 的 `FramePtr` 实现帧共享，显示、推理、录像多个消费者同时引用同一帧而不需要拷贝。录像用 FFmpeg 的 `h264_rkmpp` 硬件编码器，通过 `DRM_PRIME` 直接使用 DMA-BUF fd。录像任务用 `try_enqueue` 非阻塞投递，队列满时丢帧。

⑦ IPC 通信模块

作为 Unix Socket 服务端监听 4 个 socket: `main_face`、`main_roi`、`qt_main`、`main_device`。采用 Tag-based 协议（4 字节标签加 payload），所有 send 用 `MSG_DONTWAIT` 非阻塞，配合 `try_lock` 实现 fire-and-forget。主循环用 peek-then-consume 模式异步消费 ack 消息。

(2) 人脸识别进程（`face_process`）

基于 InspireFace 框架运行在 NPU 上，包含以下功能：

① V4L2 USB 摄像头采集

通过 V4L2 MMAP 方式采集 `/dev/video21` 的 YUYV 格式画面（640×360@30fps），通过 `VIDIOC_EXPBUF` 导出 DMA-BUF fd，实现零拷贝。

② InspireFace 人脸识别

加载 `Gundam_RK3588` 模型，Session 做 `FaceDetectAndTrack` 和 `FaceFeatureExtract`。特征库用 SQLite 持久化（`face_database.db`），搜索阈值 0.48，全量搜索模式。用 `FaceModule` 的 `mutex` 保证 NPU 串行访问，避免两个线程同时调用 Session。

③ RGA 零拷贝转换

USB 摄像头采集的 YUYV 格式通过 RGA 硬件转换成 BGR。通过 `wrapbuffer_fd` 包装 DMA-BUF fd, `improcess` 执行转换，不需要 CPU 拷贝。RTSP 流识别时 OpenCV 解码直接得到 BGR，跳过 RGA。

④ 识别确认与结果回传

USB 摄像头识别需要 5 帧连续确认同一人，RTSP 流识别需要 3 帧。识别结果通过 Unix Socket 回传给主进程或 QT，同时写入 `records.db` 的考勤表或设备使用表。

⑤ DisplaySender 异步推帧

独立线程加 `condition_variable` 实现的推帧队列，最多保留 3 帧。主线程采集一帧后立即 `push` 到队列异步发送，同时用虚拟地址做推理，推帧和推理完全并行。

(3) 设备进程 (device_process)

设备进程作为物联网网关，内部有两个独立管理模块，分别用不同协议管理不同设备：

① IotDevice 基类

抽象了 5 个纯虚函数：`connect`、`disconnect`、`refresh`、`getStatus`、`setPower`。基类提供事件回调、在线状态管理、基础信息 `getter`。遵循开闭原则，新增设备类型只需继承此类。

② DeviceManager 模块（管理米家插座）

MiotPlug 继承 IotDevice，通过 `miio` LAN 协议（TCP 54321）直连米家智能插座 3。`miio` 协议包括 Hello 握手（UDP）、AES-128-CBC 加密的 RPC 调用、MD5 校验和、`device_ts` 同步。DeviceManager 负责 5 秒轮询状态刷新、软件倒计时（用 `std::map` 维护到期时间戳）、过载过温保护（2000W/75°C 阈值加 60 秒冷却）。采用 lock-free RPC 模式：锁内只 `findDevice` 拿 `shared_ptr`，锁外做 RPC，避免阻塞 QT 查询。启动时执行安全策略关闭所有插座。

③ Esp32Manager 模块（管理 ESP32 摄像头）

通过 MQTT 协议调度 ESP32-S3 摄像头。不继承 IotDevice，是独立的任务调度器。管理识别任务状态机（`kWaitReady` 到 `kRecognizing` 到 `kDone` 或 `kTimeout`），30 秒超时自动取消。MQTT 订阅 `room/+/req/#`、`room/+/event/#`、`room/+/heartbeat`

三个 topic。识别成功后通过回调通知 DeviceIpcClient 转发给主进程。ESP32 心跳 60 秒超时标记离线。

④ DeviceIpcClient 模块

作为 Unix Socket 客户端同时连接 QT 和主进程的服务端，维护两条独立 IPC 通道。采用 Tag-based 协议和 MSG_WAITALL 阻塞读取，所有发送用 MSG_DONTWAIT 非阻塞，tag 加 payload 合并为单次 send 保证原子性。handleQtConnection 处理插座查询、添加删除和 ESP32 管理命令。

⑤ MqttClient 模块

封装 paho-mqtt-cpp 异步客户端，断线自动重连，遗嘱消息通知 ESP32 设备进程掉线。

(4) QT 界面进程 (qt_face_ui)

基于 Qt5 加 OpenGL 实现，支持双屏输出，包含以下功能：

① GLStreamView OpenGL 渲染

继承 QOpenGLWidget，支持 NV12 和 YUYV 两种格式。通过 mmap DMA-BUF fd 拿到数据后用 glTexSubImage2D 上传纹理，YUV 到 RGB 转换用 BT.601 矩阵的片段着色器。检测框用 QPainter 叠加绘制。

② FrameReceiver 帧接收

作为 Unix Socket 服务器等待主进程和人脸进程连接。通过 recvmsg 的 SCM_RIGHTS 辅助数据接收 DMA-BUF fd，实现跨进程零拷贝传递。两个接收器分别收主进程 NV12 画面和人脸进程 YUYV 画面。

③ 多路画面显示

支持 8 路画面同时显示，2×4 网格模式或单路全屏模式切换。模式切换时主动销毁重建 GLStreamView 保证 OpenGL 上下文干净。每路画面叠加 ROI 状态色框（绿色空闲、红色使用中）。

④ ROI 配置可视化编辑

RoiEditView 继承 GLStreamView，鼠标拖拽绘制 ROI 区域，支持撤销操作。保存时写回 roi.conf 文件，同时通过 socket 通知主进程热重载。

⑤ 设备管理与手动断电

QTabWidget 分智能插座和 ESP32 摄像头两个标签页，2 秒自动刷新状态。

手动断电走 QT 到 main 到 device 的中转链路：QT 发 MainCmd 给主进程，主进程清检测状态后转发 DeviceEvent 给设备进程执行断电。

⑥ 系统仪表盘

通过读 proc 和 sys 文件系统显示 CPU、内存、NPU、RGA、DDR、GPU、VPU 和温度 8 类指标，2 秒刷新。

(5) 共享基础设施

四进程共享的底层支撑组件：

① SQLite 数据库

两个数据库：records.db 用 WAL 模式加 busy_timeout=3000ms 实现多进程并发安全，存储插座配置、ESP32 信息、考勤记录、ROI 配置等；face_database.db 由 InspireFace SDK 管理，存储人脸特征向量。

② Unix Socket IPC 协议

所有进程间通信基于 Unix 域套接字。Tag-based 协议用 4 字节标签区分消息类型。DMA-BUF fd 通过 sendmsg 的 SCM_RIGHTS 辅助数据跨进程传递。

③ MQTT Broker

运行在 RK3588 上的 mosquitto（端口 1883），作为 ESP32 和设备进程间的消息中间件，支持断线自动重连和遗嘱消息。

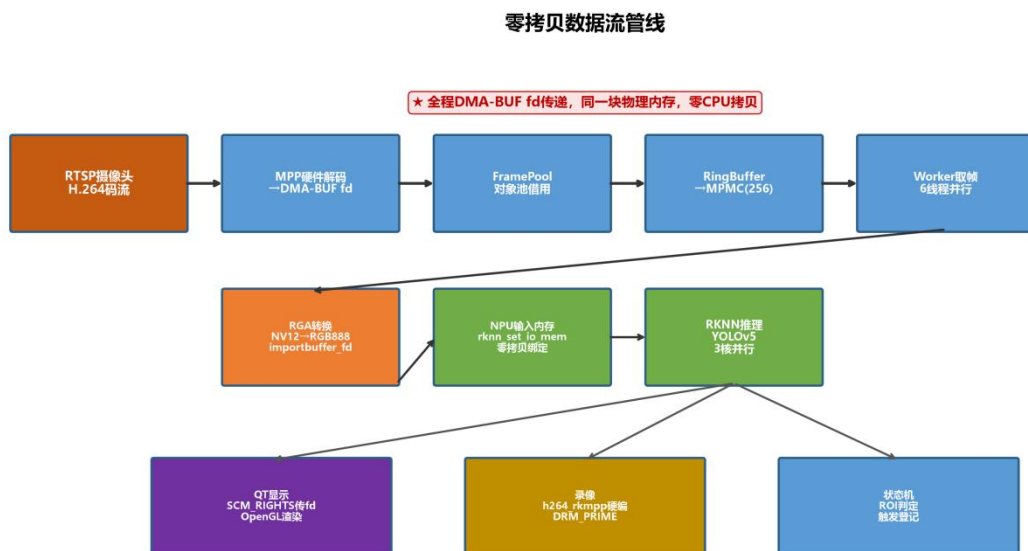


图 2.10 零拷贝数据流管线

被动设备登记业务流程图



图 2.11 被动设备登记业务流程图

3、 完成情况及性能参数

3.1 整体介绍



图 3.1 系统俯视图



图 3.2 系统正面图

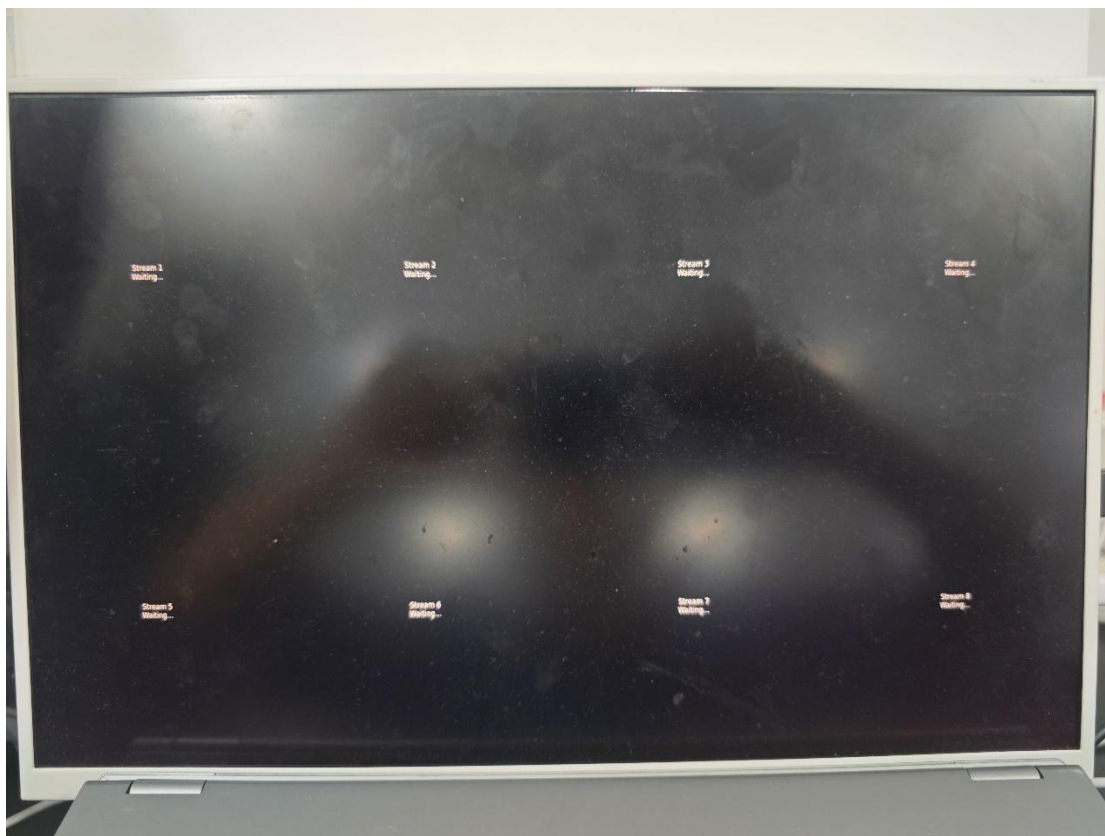


图 3.3 HDMI 显示屏默认界面(没接入网络视频流)

系统整体图如图 3.1、3.2、3.3 所示，包括了 RK3588ELF2，USB 摄像头，7 寸 MIPI 触摸屏以及一个 HDMI 显示屏。此外，还使用了 ESP32S3N16R8 搭配 0V2640 作为下位机。

触摸屏显示 QTUI 界面与用户进行交互，下位机用来辅助监控画面做每个房间的设备使用登记，HDMI 用来显示 8 路 1080p 视频流拉流解码叠加推理框的画面，USB 摄像头用来主动考勤和设备登记使用。

3.2 工程成果

3.2.1 机械成果



图 3.4 3D 打印框架图

我们使用了 3D 打印技术得到了整个系统的外部框架如图 3.4 所示，其中各项功能如图所示。

3.2.2 电路成果

如图，包括了 RK3588 供电模块，子机 ESP32s3 以及其触摸屏 ILI9341 模块等。

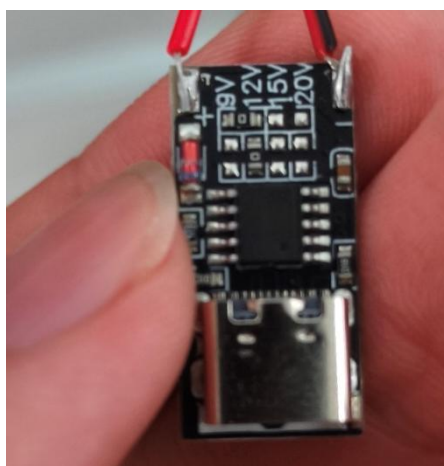


图 3.5 RK3588 散热风扇供电模块

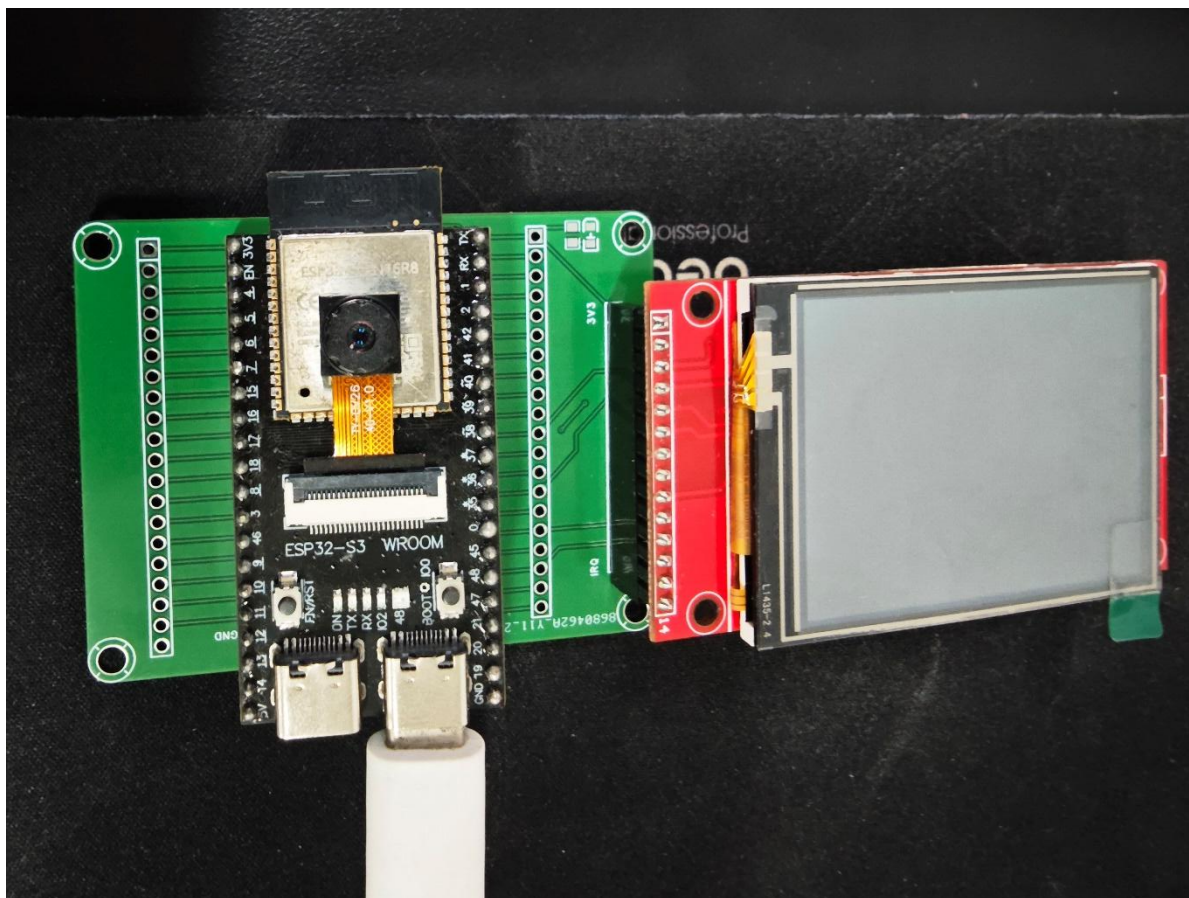


图 3.6 ESP32s3 以及 ILI9341 触摸屏

3.2.3 软件成果

我们建立了整个管理系统，主要包括了管理系统和 HDMI 显示屏监视部分，具体图见图 3.7、3.8 所示。



图 3.7 HDMI 显示屏监视图

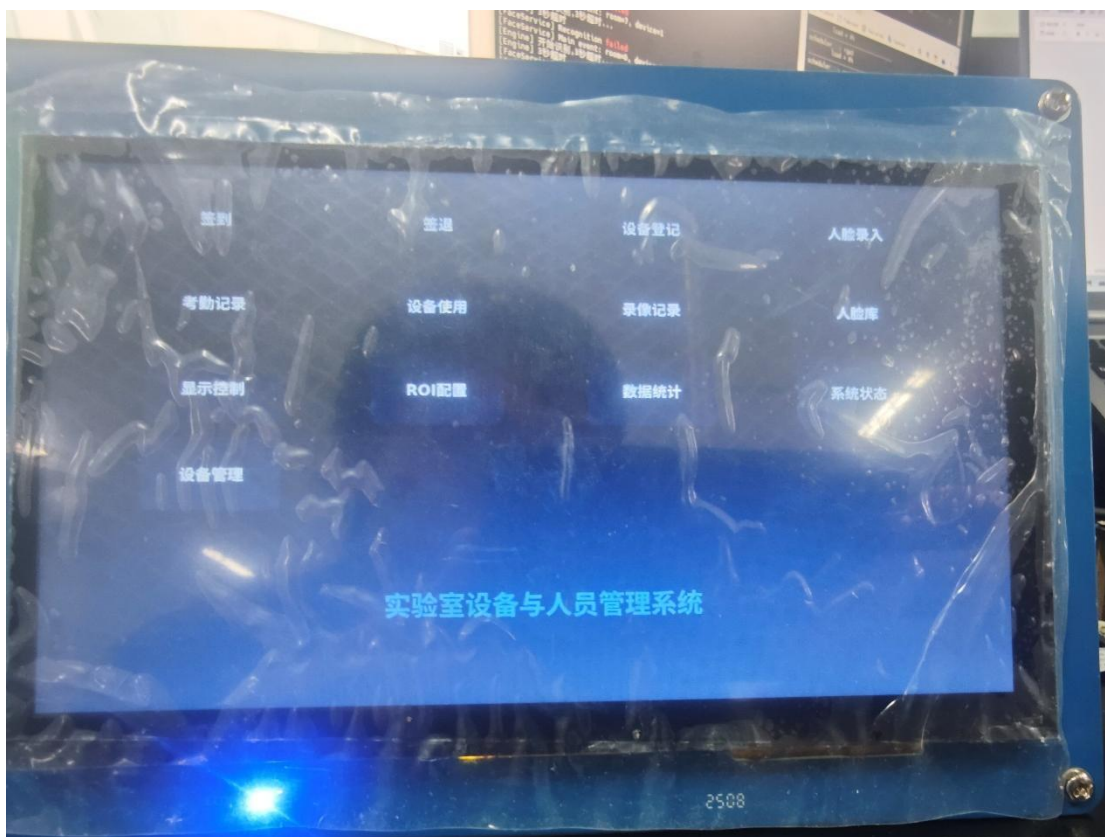


图 3.8 触摸屏显示 QT 交互界面图

3.3 特性成果

(1) 系统主页，上电显示的初始界面，包含了各项功能,如图 3.9 所示。

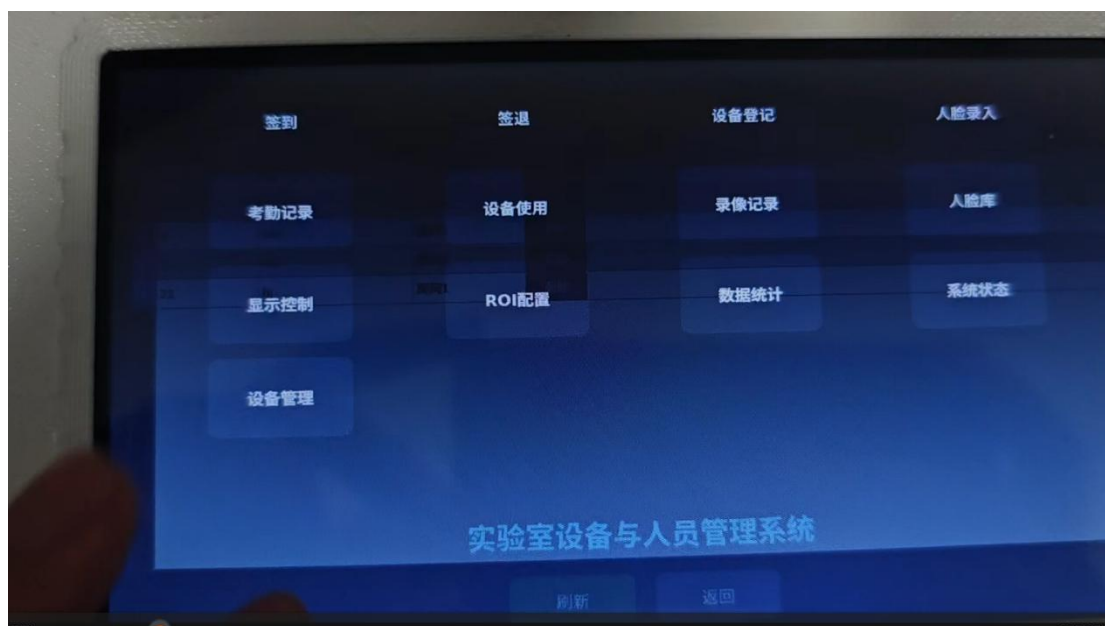


图 3.9 系统主页图

(2) 通过摄像头识别 USB 摄像头的人脸，并借此进行签到签退和和设备使用登记的记录，如图 3.10 所示。

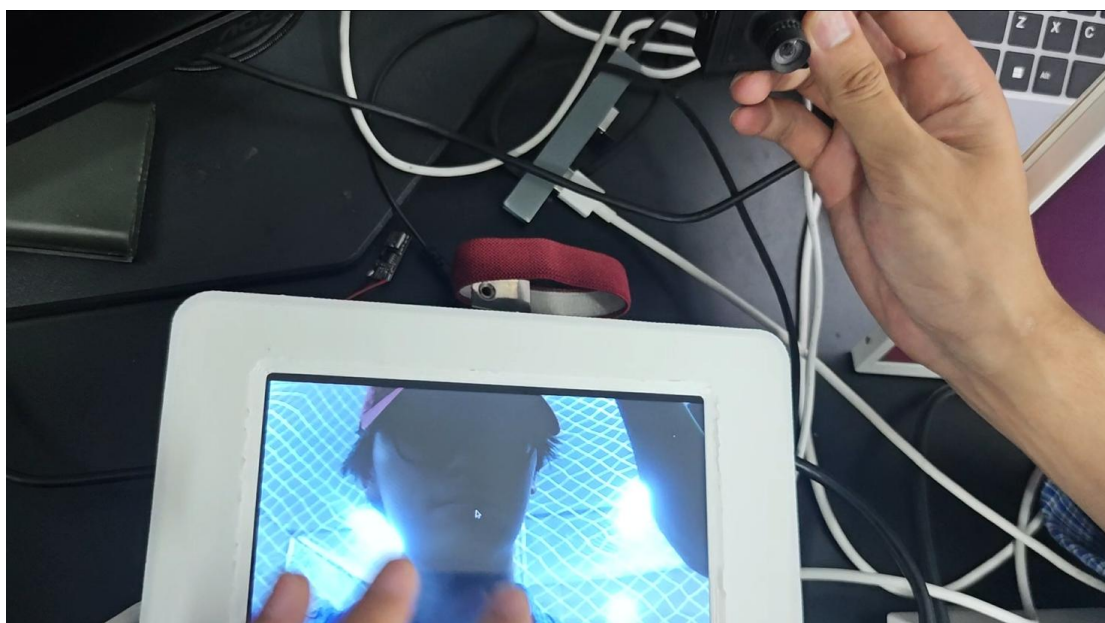


图 3.10 人脸系统识别图

(3) 可以对录入的人脸进行管理，未人脸库的人脸不予识别。只有人脸库中的人脸信息才可以进行签到签退和设备使用登记的正确识别，人脸库管理界

面如图 3.11。

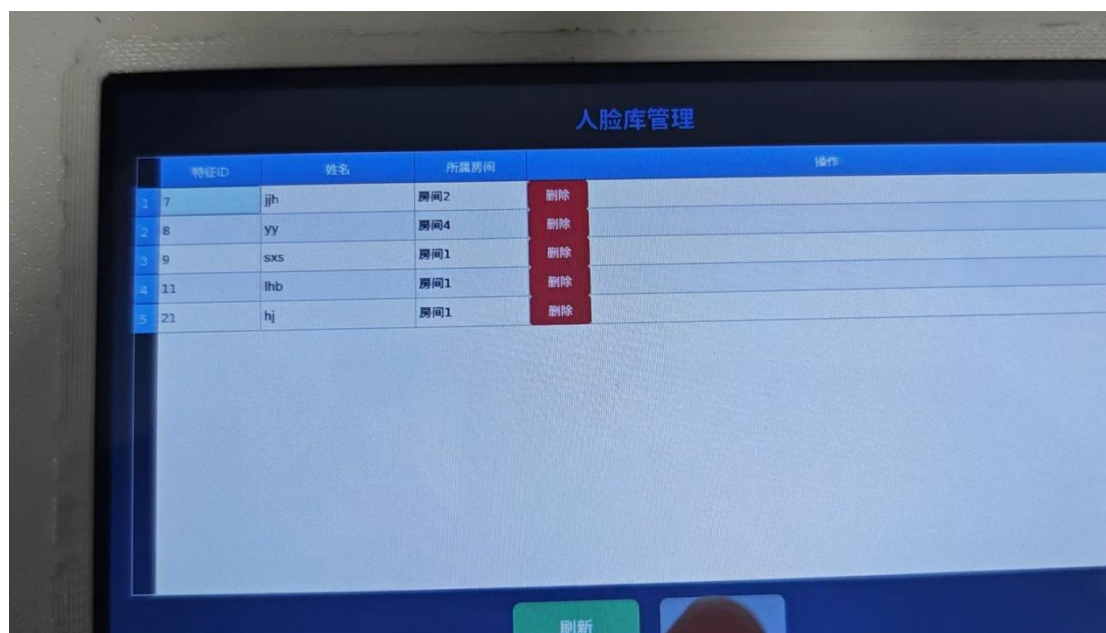


图 3.11 人脸库管理系统图

(4) 可以记录签到签退，建立考勤记录表，对已录入的人脸信息进行考勤管理，如图 3.12。

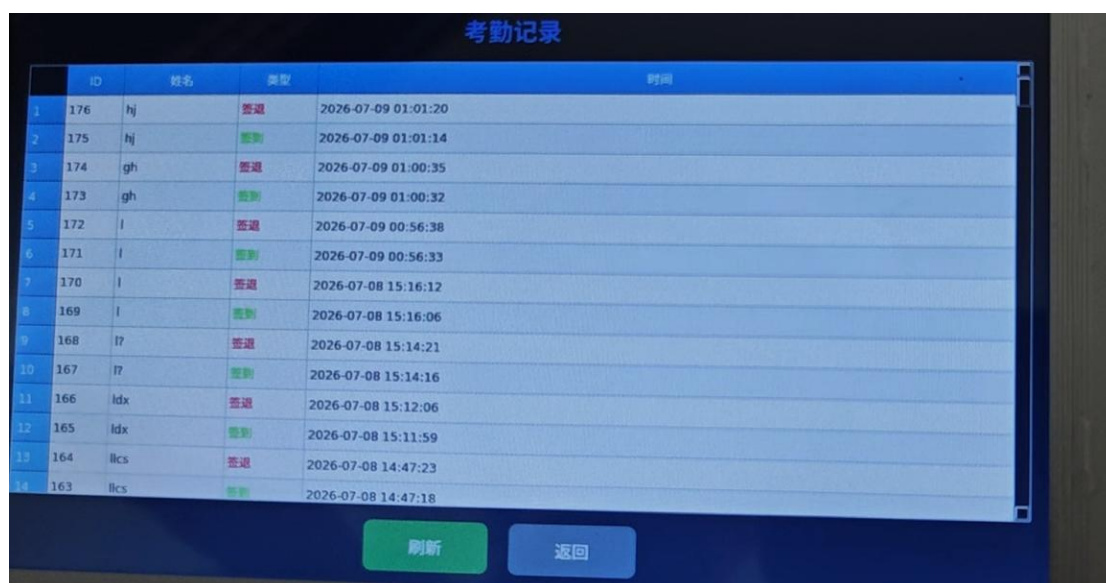


图 3.12 考勤记录系统图

(5) 可以记录各个房间中的设备使用情况，建立设备使用记录表，记录各个设备具体使用情况，如图 3.13。

	ID	姓名	房间号	设备号	时长(分)	时间
1	322	I	0	1	15	2026-07-08 16:15:06
2	321	I	0	2	15	2026-07-08 16:14:54
3	320	I	0	1	15	2026-07-08 16:13:45
4	319	I	0	1	15	2026-07-08 16:01:52
5	318	I	0	1	15	2026-07-08 16:00:57
6	317	I	0	1	15	2026-07-08 16:00:30
7	316	I	0	1	15	2026-07-08 15:59:31
8	315	I	0	1	15	2026-07-08 15:49:12
9	314	IT	0	1	15	2026-07-08 15:14:32
10	313	idx	0	1	15	2026-07-08 15:12:16
11	312	sxs	0	1	15	2026-07-08 15:04:36
12	311	sxs	0	2	15	2026-07-08 14:52:33
13	310	lhb	0	1	15	2026-07-08 14:37:08
14	309	lc	0	1	15	2026-07-08 14:30:13

图 3.13 设备使用记录图

(6) 对每一路摄像头的各个设备区域，如果有人和设备 ROI 区域内时间超阈值，那么开始录像直到没有人在对应设备区域时间超阈值，停止录像。本地进行分别录像并储存，并支持播放其中单个录像，如图 3.14、3.15 所示。

	ID	通道	开始时间	结束时间	录像文件	操作
1	3531	4	20260709_090408	20260709_090411	stream3_20260709_090408....	播放
2	3528	8	20260709_090401	20260709_090404	stream7_20260709_090401....	播放
3	3527	7	20260709_090359	20260709_090402	stream6_20260709_090359....	播放
4	3529	4	20260709_090359	20260709_090404	stream3_20260709_090359....	播放
5	3530	6	20260709_090358	20260709_090405	stream5_20260709_090358....	播放
6	3526	4	20260709_090348	20260709_090351	stream3_20260709_090348....	播放
7	3524	7	20260709_090347	20260709_090350	stream6_20260709_090347....	播放
8	3522	4	20260709_090341	20260709_090344	stream3_20260709_090341....	播放
9	3523	8	20260709_090341	20260709_090344	stream7_20260709_090341....	播放
10	3525	3	20260709_090339	20260709_090351	stream2_20260709_090339....	播放
11	3521	4	20260709_090335	20260709_090338	stream3_20260709_090335....	播放
12	3519	3	20260709_090331	20260709_090336	stream2_20260709_090331....	播放
13	3520	1	20260709_090329	20260709_090336	stream0_20260709_090329....	播放
14	3515	7	20260709_090325	20260709_090328	stream6_20260709_090325....	播放

图 3.14 8 路摄像头录像记录系统图

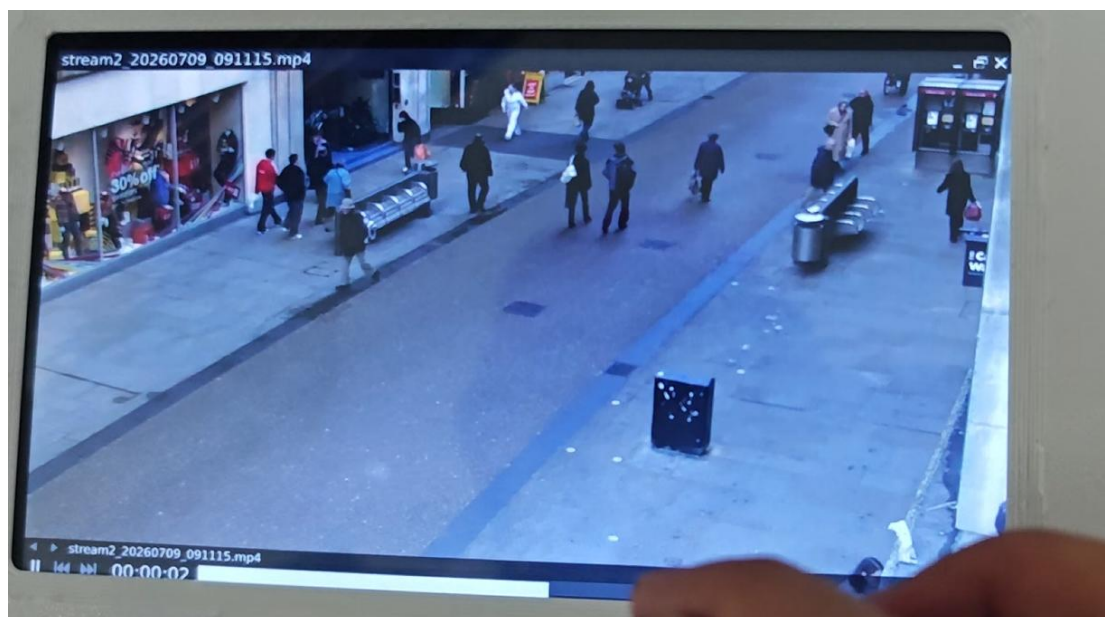


图 3.15 录像系统播放录像图

(7) 考虑到每个房间的设备位置可能会发生变化，对于每一路摄像头，可以配置其每个设备的 ROI 区域，用来进行设备使用 ROI 区域监测。同时还支持动态配置和热加载配置，不用重启系统。ROI 配置图如图 3.16 所示，显示效果如图 3.17 所示。



图 3.16 ROI 区域配置图



图 3.17 不同区域的不同 ROI 配置图

(8) 支持对设备进行管理，可以查看已经配置的米家智能插座 3 的设备情况，比如实时使用功率，是否开关打开，设备在线与否，设备温度等信息，支持远程断电。还可以查看已经配置的每个房间一个的 ESP32 设备情况。两种设备均支持手动配置添加新设备。如图 3.18、3.19 所示。

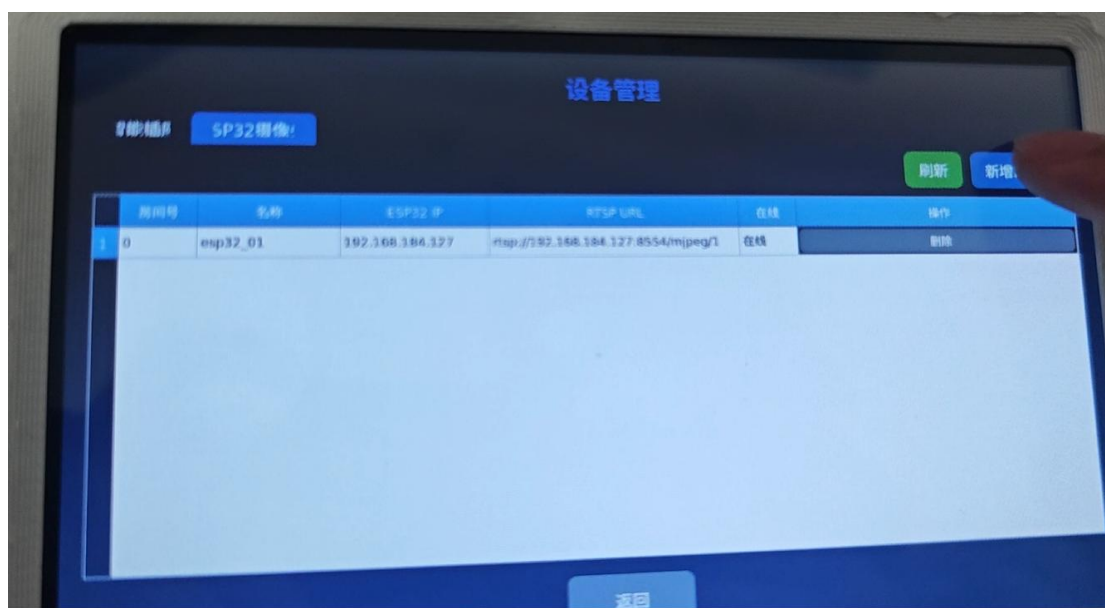


图 3.18 ESP32 设备管理系统图

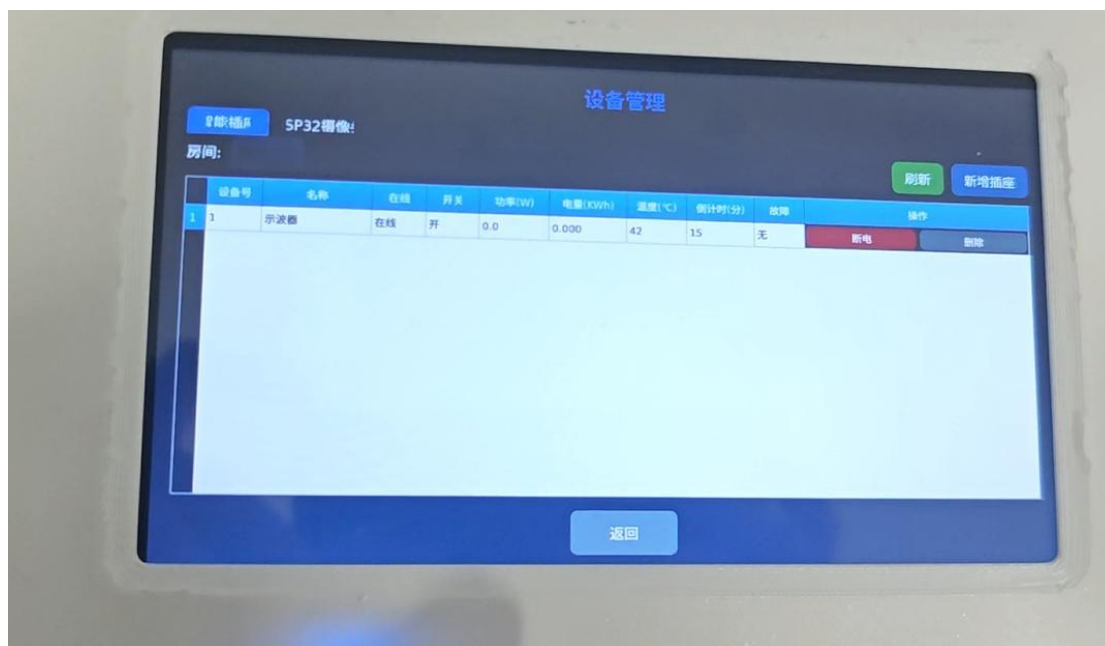


图 3.19 智能插座管理系统图

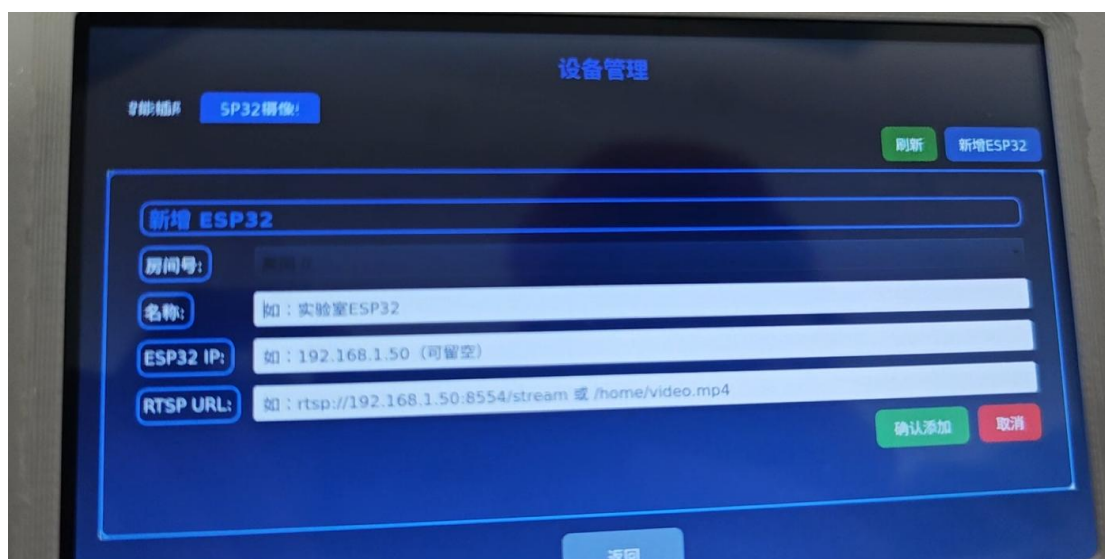


图 3.20 ESP32 设备添加界面图

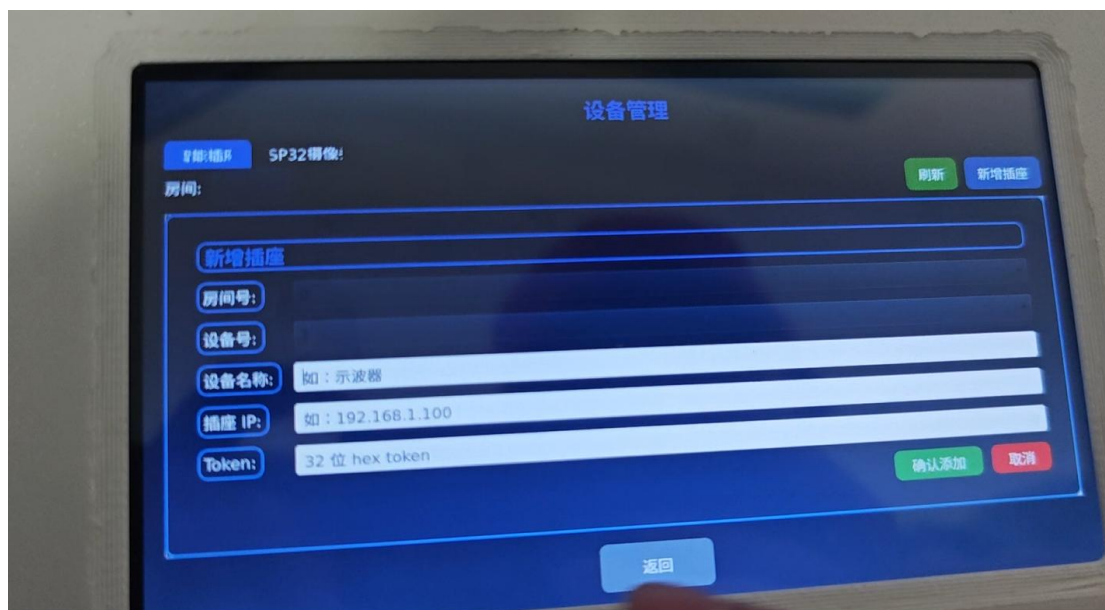


图 3.21 智能设备添加界面图

(9) 支持对考勤和设备进行可视化处理，对于考勤记录，使用柱形图显示，支持不同房间和不同周期进行筛选显示；对于设备使用，使用饼状图、显示出每个设备的使用量占比。同时，支持按时间、房间等多维度进行筛选。统计图见图 3.21、3.22 所示。



图 3.22 考勤数据统计图

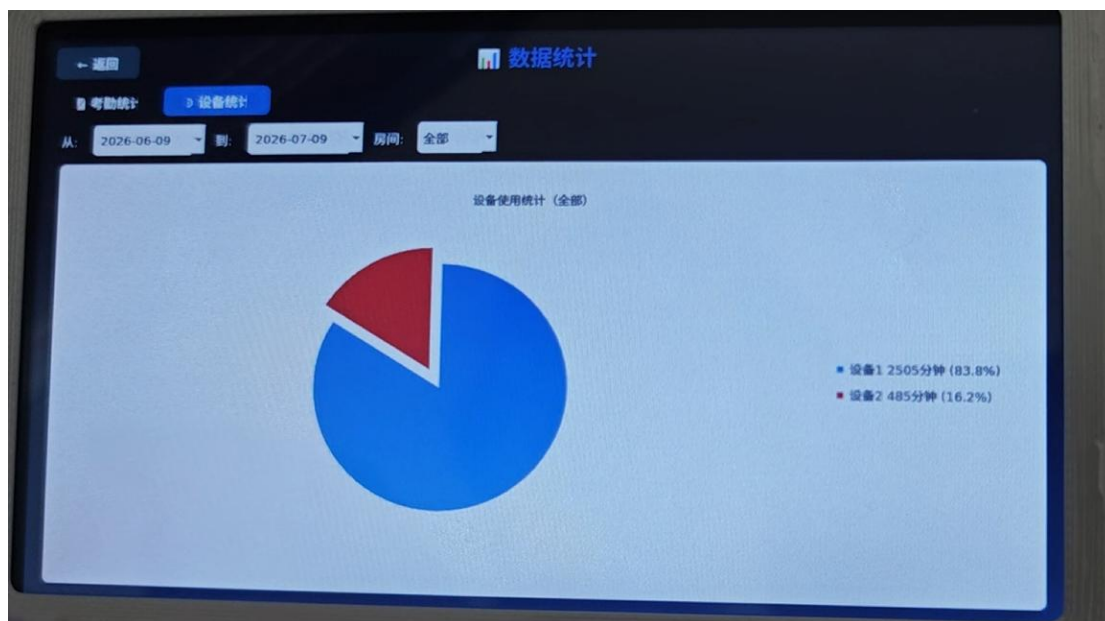


图 3.23 设备使用情况饼状图

(10) 支持对整个系统状态进行实时监测，可以直观地显示各项资源实时占用量，比如 CPU 占用和温度，以及 RGA,NPU,GPU,内存和带宽等占用情况，方便对系统资源进行统筹规划，系统状态图见图 3.22。



图 3.24 正常运行时系统状态图

4、 总结

4.1 可扩展之处

(1) 多房间扩展。当前系统以房间 0 为试点，数据库和 MQTT topic 都按 room_id 设计，可以直接扩展到多房间多 ESP32 节点。

(2) 算法升级。NPU 推理框架支持模型热替换，可以升级到 YOLOv8 或换其他检测模型。也可以选择接入其他模型丰富功能，如火焰检测和跌倒检测等。

(3) 云端联动。MQTT 协议可以对接云端平台，实现远程监控和数据汇聚。

(4) 移动端 APP。可以开发移动端 APP 通过 MQTT 接入，实现远程设备管理和考勤查询。

(5) 更多 IoT 设备。IotDevice 基类设计遵循开闭原则，新增设备类型只需继承实现 5 个虚函数。

(6) 能耗分析。基于插座的电量采集和使用时长记录，可以增加能耗统计和可视化分析模块。

4.2 心得体会

本项目是我在嵌入式 Linux 与音视频领域的一次综合性工程实践，从需求分析到架构设计、从核心模块开发到多设备联调，完整经历了一个嵌入式 AI 系统的全生命周期。在开发过程中，我深刻体会到以下几点：

本项目是我在嵌入式 Linux 和音视频领域的一次综合工程实践。从需求分析到架构设计，从核心模块开发到多设备联调，完整经历了一个嵌入式 AI 系统的全生命周期。过程中踩了不少坑，也有不少收获。

第一，零拷贝是嵌入式 AI 视觉系统的性能基石。RK3588 有强大的 NPU 和硬件加速器，但如果数据通路里存在 CPU 拷贝，再强的算力也会被内存带宽和 CPU 负载拖垮。项目最初用传统方式做，解码后 CPU 拷贝、软件色彩转换、再拷贝喂 NPU，CPU 占用率超过 70%，帧率也不稳定。改成 DMA-BUF fd 传递以后，CPU 占用降到 30%以下，帧率稳定在 25fps。这次优化让我明白，嵌入式系统的性能优化不能只盯着算法，数据通路同样关键。

第二，多进程架构的 IPC 设计最容易出问题。项目四个进程通过 Unix Socket 通信，一开始用 MSG_PEEK 按缓冲区长度区分消息类型，结果两条消息合并到

达时长度对不上，直接死循环，级联引发推理管线卡死。排查这个问题花了很多时间，最后改成 Tag-based 协议才彻底解决。这让我深刻认识到，Unix Socket 是流式协议没有消息边界，应用层必须用长度前缀或类型标签来显式标记边界，非阻塞 I/O 还要配合 peek-then-consume 模式避免半读问题。

第三，多设备协调需要严谨的状态机。ESP32 被动识别流程涉及用户确认、30 秒超时、推流启停、识别结果回传等多个异步环节，没有状态机管理很容易状态混乱。我设计了 kWaitReady 到 kRecognizing 到 kDone 或 kTimeout 的状态流转，配合超时自动取消和双重通知机制，保证了流程的可靠性。

第四，调试嵌入式多进程系统要有一套系统化的方法。项目里最棘手的 bug 是设备进程一启动主进程就卡死，表现是 MPMC 队列满和 HDMI 画面冻屏。四个进程交互，单点调试很难定位。我通过日志时间线对照、反向追踪数据流、分析锁竞争关系，最终定位到设备进程的 IPC 死循环。这次经历让我掌握了从现象反向追踪根因的方法。

第五，工程规范比功能实现更重要。项目里遇到的很多问题，比如 SIGPIPE 崩溃、SQLite 锁冲突、插座默认上电，本质上都是工程规范缺失导致的。signal(SIGPIPE, SIG_IGN)、PRAGMA journal_mode=WAL、安全默认策略，这些不起眼的细节才是系统稳定运行的保障。

通过这个项目，我不仅掌握了 RK3588 的 NPU 开发、GStreamer 管线编程、多进程 IPC 设计、IoT 协议对接这些具体技术，更重要的是建立了嵌入式 AI 系统的工程化思维。架构先行、协议严谨、非阻塞设计、安全默认，这些经验会对我以后的工程实践产生长远影响。

5、参考文献

- [1] 韦东山. 嵌入式 Linux 应用开发完全手册 [M]. 北京: 人民邮电出版社, 2022.
- [2] 瑞芯微电子股份有限公司. RK3588 NPU 开发编程指南 [EB/OL]. (2022-08-01)[2025-04-20]. <https://www.rock-chips.com/a/cn/downloadcenter>.
- [3] 王洪辉, 张何非, 庾先国. 基于 DMA-BUF 的嵌入式视频全链路零拷贝技术 [J]. 计算机工程与科学, 2021, 43 (8): 1375-1381.
- [4] 汪草. GStreamer 音视频开发实战 [M]. 北京: 电子工业出版社, 2023.
- [5] Redmon J, Farhadi A. YOLOv5: Unified Real-Time Object Detection [EB/OL]. (2020-06-25)[2025-03-15]. <https://github.com/ultralytics/yolov5>.
- [6] 李立宗. 人脸识别原理与工程实现 [M]. 北京: 机械工业出版社, 2022.
- [7] 吴功宜, 吴英. 物联网与智能设备通信技术 [M]. 3 版. 北京: 清华大学出版社, 2022.
- [8] 陈莉君, 康华. Linux 内核原理与实现 [M]. 3 版. 北京: 机械工业出版社, 2021.
- [9] SQLite 官方开发团队. SQLite Write-Ahead Logging 技术文档 [EB/OL]. (2024-01-12)[2025-04-10]. <https://www.sqlite.org/wal.html>.